

EXPERIMENT :- 7

OBJECT :- WAP to retrain a pretrained imagenet model to classify a medical image dataset.

THEORY :- We are training a **CNN model from scratch using ResNet18** architecture on **CIFAR-10** to classify images into 10 classes.

“We trained a modified ResNet18 model on CIFAR-10 dataset. We applied data augmentation and normalization, used CrossEntropy loss and Adam optimizer, and improved training using OneCycle learning rate scheduler. The model learns image features and classifies them into 10 categories. Performance is evaluated using accuracy, loss graphs, and confusion matrix.”

COMPONENT	ROLE
CNN	Extracts image features
ResNet	Uses skip connections
Data Augmentation	Improves generalization
Optimizer	Updates weights
Scheduler	Adjusts learning rate
Confusion Matrix	Performance analysis

Model performance measured using:

- Accuracy
- Loss
- Confusion Matrix
-

This Model is Good for Medical Images

- Works well with small datasets
- Saves training time

Learns complex patterns
High accuracy with fine-tuning

What is a Pretrained ImageNet Model?

- ImageNet is a large dataset with **millions of images and 1000 classes**
- Models trained on it learn:
 - edges
 - textures
 - shapes
 - patterns

These features are **generic**, so we reuse them for medical images.

CNN (Convolutional Neural Network)

Extracts important features from images such as edges, textures, and patterns.

ResNet (Residual Network)

Uses skip connections to avoid vanishing gradient and enables training of deep networks.

Data Augmentation

Applies transformations (flip, crop, rotation) to increase dataset size and improve generalization.

Optimizer

Updates model weights during training to minimize loss (e.g., Adam).

Shecduler

Dynamically adjusts the learning rate to improve training performance.

Confusion Matrix

Evaluates model performance by comparing predicted vs actual classes.

CODE :-

```
# =====
```

1. Imports

```
# =====
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
import torchvision
```

```
import torchvision.transforms as transforms
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
print("Using device:", device)
```

```
# =====
```

2. Fast Transforms (NO 224!)

```
# =====
```

```
transform_train = transforms.Compose([
```

```
    transforms.RandomHorizontalFlip(),
```

```
    transforms.RandomCrop(32, padding=4),
```

```
    transforms.ToTensor(),
```

```
    transforms.Normalize((0.5,0.5,0.5), (0.5,0.5,0.5))
```

```
])
```

```
transform_test = transforms.Compose([
```

```
    transforms.ToTensor(),
```

```
    transforms.Normalize((0.5,0.5,0.5), (0.5,0.5,0.5))
```

])

=====

3. Dataset

=====

```
train_dataset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True,
transform=transform_train)
```

```
test_dataset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True,
transform=transform_test)
```

```
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=128, shuffle=True)
```

```
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=128, shuffle=False)
```

=====

4. Model (ResNet18 Modified)

=====

```
model = torchvision.models.resnet18(weights=None) # train from scratch (faster for CIFAR)
```

```
model.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
```

```
model.maxpool = nn.Identity()
```

```
model.fc = nn.Linear(512, 10)
```

```
model = model.to(device)
```

=====

5. Loss + Optimizer + Scheduler

=====

```
criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(model.parameters(), lr=0.001)

scheduler = torch.optim.lr_scheduler.OneCycleLR(

    optimizer,

    max_lr=0.01,

    steps_per_epoch=len(train_loader),

    epochs=15

)

# =====

# 6. Training

# =====

epochs = 15

train_losses = []

train_accuracies = []

for epoch in range(epochs):

    model.train()

    running_loss = 0

    correct = 0

    total = 0

    for images, labels in train_loader:

        images, labels = images.to(device), labels.to(device)
```

```
optimizer.zero_grad()

outputs = model(images)

loss = criterion(outputs, labels)

loss.backward()

optimizer.step()

scheduler.step()
```

```
running_loss += loss.item()

_, predicted = outputs.max(1)

total += labels.size(0)

correct += predicted.eq(labels).sum().item()
```

```
acc = 100 * correct / total

loss_epoch = running_loss / len(train_loader)
```

```
train_losses.append(loss_epoch)

train_accuracies.append(acc)
```

```
print(f"Epoch {epoch+1}: Loss={loss_epoch:.4f}, Accuracy={acc:.2f}%")
```

```
# =====
```

```
# 7. Graphs
```

```
# =====
```

```
plt.plot(train_accuracies)

plt.title("Accuracy vs Epoch")

plt.xlabel("Epoch")
```

```
plt.ylabel("Accuracy")
```

```
plt.show()
```

```
plt.plot(train_losses)
```

```
plt.title("Loss vs Epoch")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
```

```
plt.show()
```

```
# =====
```

```
# 8. Confusion Matrix
```

```
# =====
```

```
model.eval()
```

```
all_preds = []
```

```
all_labels = []
```

```
with torch.no_grad():
```

```
    for images, labels in test_loader:
```

```
        images = images.to(device)
```

```
        outputs = model(images)
```

```
        _, preds = outputs.max(1)
```

```
        all_preds.extend(preds.cpu().numpy())
```

```
        all_labels.extend(labels.numpy())
```

```
cm = confusion_matrix(all_labels, all_preds)
```

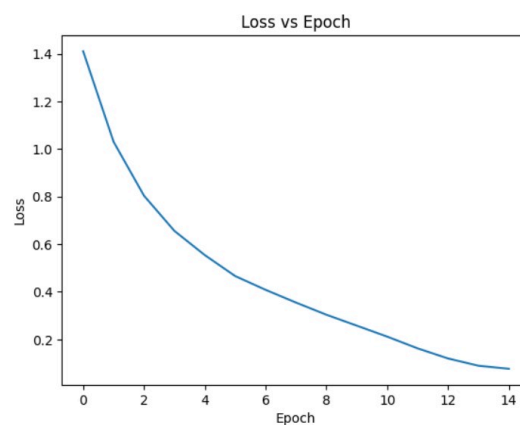
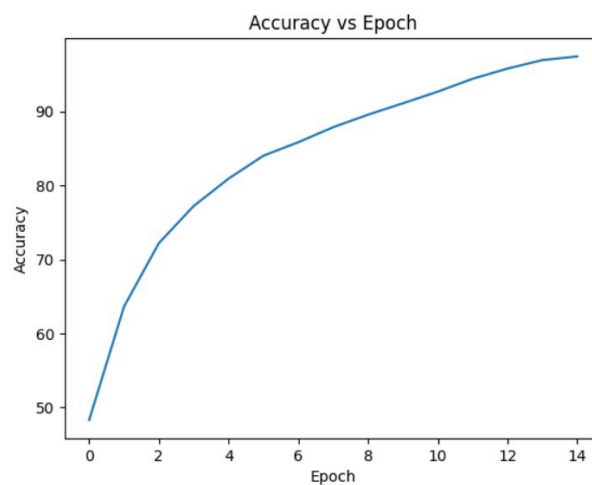
```
ConfusionMatrixDisplay(cm).plot()
```

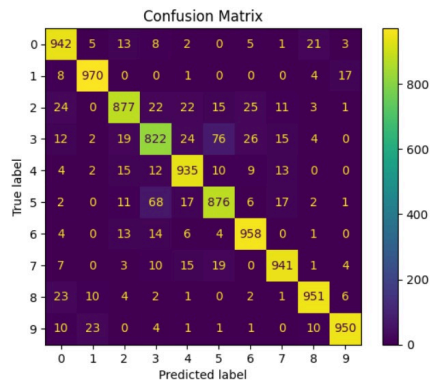
```
plt.title("Confusion Matrix")
```

```
plt.show()
```

OUTPUT :-

```
Using device: cuda
Epoch 1: Loss=1.4104, Accuracy=48.33%
Epoch 2: Loss=1.0303, Accuracy=63.63%
Epoch 3: Loss=0.8037, Accuracy=72.21%
Epoch 4: Loss=0.6560, Accuracy=77.23%
Epoch 5: Loss=0.5542, Accuracy=80.92%
Epoch 6: Loss=0.4661, Accuracy=84.02%
Epoch 7: Loss=0.4084, Accuracy=85.84%
Epoch 8: Loss=0.3548, Accuracy=87.87%
Epoch 9: Loss=0.3037, Accuracy=89.56%
Epoch 10: Loss=0.2578, Accuracy=91.09%
Epoch 11: Loss=0.2117, Accuracy=92.66%
Epoch 12: Loss=0.1624, Accuracy=94.40%
Epoch 13: Loss=0.1200, Accuracy=95.78%
Epoch 14: Loss=0.0895, Accuracy=96.93%
Epoch 15: Loss=0.0767, Accuracy=97.42%
```





Conclusion :-

The model effectively learns image features using a CNN-based ResNet architecture and achieves good classification accuracy. With proper training techniques and evaluation methods, it demonstrates the effectiveness of deep learning in image classification tasks.